DCC006/DCC017 Organização de computadores I/Introdução à Arquitetura de Computadores

Trabalho Prático #1

Professores: Daniel Fernandes Macedo e Omar Paranaiba Vilela Neto

Antes de começar seu trabalho, leia todas as instruções abaixo.

- O trabalho deve ser feito individualmente. Cópias de trabalho acarretarão em devida penalização às partes envolvidas.
- Entregas após o prazo serão aceitas, porém haverá uma penalização. Quanto maior o atraso maior a penalização.
- **Submeta apenas um arquivo .zip contendo suas soluções e um relatório em formato .pdf, com no máximo duas páginas, nomeado com seu nome completo e número de matrícula. O relatório deve descrever brevemente o que foi feito no trabalho, por exemplo, as decisões de implementação tomadas, dificuldades enfrentadas, testes realizados e os principais aprendizados adquiridos**. Os arquivos de código devem ser nomeados de acordo com a numeração do problema correspondente (por exemplo, prob1.s para o problema 1).
- O objetivo do trabalho é praticar as suas habilidades na linguagem assembly. Para isso, você utilizará o *Venus Simulator* (https://www.kvakil.me/venus/). O Venus é um simulador de ciclo único que te permite enxergar o valor armazenado em cada registrador e seguir a execução do seu código linha a linha. O simulador foi desenvolvido por Morten Petersen e possui a ISA do RISC-V, embora apresente algumas alterações. Você pode utilizar o seguinte link: https://github.com/mortbopet/Ripes/blob/master/docs/introduction.md para verificar as modificações da sintaxe ISA utilizada pelo simulador. Note que no livro e material da disciplina os registradores são de 64 bits, mas o simulador utiliza registradores de apenas 32 bits. Para utilizar o simulador basta você digitar seu código aba *Editor* e para executá-lo basta utilizar a aba *Simulator.*
- A correção do trabalho prático usará o simulador, e será feita de forma automatizada. Portanto, é crucial que vocês **empreguem as convenções de chamada de procedimento e de gerenciamento de pilha do RISC-V**. Isso irá permitir que o trabalho desenvolvido seja avaliado corretamente (por exemplo, iremos usar registradores “sx”, que são salvos pelo chamador, para realizar a contabilização automática de testes que foram executados corretamente. em outras palavras, caso seu procedimento use registradores "sx" eles deverão ser salvos na pilha). Para cada uma das questões, iremos definir um arquivo de base, onde está marcado a partir de qual ponto vocês deverão fazer o seu código. A avaliação irá considerar somente o que estiver escrito dentro daqueles limites (pois no momento da correção iremos alterar o início e fim do código fonte para fazer a correção).
- Eventuais testes apresentados nesta documentação são somente para indicar a funcionalidade a ser desenvolvida. O código dos alunos deve funcionar corretamente para todo e qualquer caso, inclusive aqueles que não estão previstos nos códigos, desde que sigam as especificações do trabalho. Façam testes além dos que estão descritos nesta documentação.
- Em ambos os problemas especificados estamos fornecendo alguns exemplos de execução, para que saibam como o código será avaliado. A sugestão é que enviem o código baseado nestes exemplos, modificando apenas a região delimitada. Com isso será possível executar scripts que substituem, automaticamente, a parte exemplo do programa com o código de testes. Caso sejam feitas alterações fora destas partes, o seu programa corre o risco de funcionar parcialmente ou mesmo não funcionar.
- Dica final: O Venus não possui o apelido r0 para registrador de retorno. Usem o a0 como registrador de retorno, porque no RISC-V o a0 e o r0 mapeiam para o mesmo registrador físico.

Problema 1: Vetor em ordem crescente (prob1.s) (5 pontos)

Escreva um procedimento chamado `e_crescente` que verifica se os elementos de um vetor estão estritamente ordenados de forma crescente (sem repetições). O primeiro argumento é o vetor, e o segundo argumento é o tamanho do vetor. O procedimento deve retornar 1 no registrador `a0` se o vetor estiver em ordem crescente, ou 0 caso contrário.

Assuma que a memória onde os números do vetor serão escritos possui espaço suficiente para que todos eles sejam escritos.

Utilize o esqueleto a seguir para o seu arquivo prob1.s (repare que a parte acima e abaixo do MODIFIQUE AQUI poderá ser alterada pelo professor/monitor no momento da correção):

```
.data

##### R1 START MODIFIQUE AQUI START #####
#
# Este espaço é para você definir as suas constantes e vetores auxiliares.
#

vetor1: .word 2, 3, 4

vetor2: .word 1, 5, 4, 6, 8

##### R1 END MODIFIQUE AQUI END #####

.text
  add s0, zero, zero       # s0 armazenará o número de testes que passaram
  la a0, vetor1
  addi a1, zero, 3
  jal e_crescente
  addi t0, zero, 1
  bne a0, t0, teste_2
  addi s0,s0,1

teste_2:
  la a0, vetor2
  addi a1, zero, 5
  jal e_crescente
  addi t0, zero, 0
  bne a0, t0, FIM
  addi s0,s0,1
  j FIM

##### R2 START MODIFIQUE AQUI START #####

e_crescente:
      jalr zero, 0(ra)

##### R2 END MODIFIQUE AQUI END #####

FIM:
```

Problema 2: Atualização de valor do estoque (prob2.s) (5 pontos)

O Sr. Oswaldo, proprietário da empresa XPTO, precisa atualizar o valor real do seu estoque. Atualmente, os preços estão defasados, e ele deseja calcular o valor real do estoque com base nas seguintes regras:

1. Aplicar um fator de aumento de 3x sobre o preço original de cada item no estoque.
2. Aplicar uma taxa adicional de 50% (relacionada a impostos) sobre o valor ajustado.

Para ajudá-lo, você deverá implementar três procedimentos em assembly, responsáveis por ajustar os preços e calcular o valor total do estoque atualizado. São eles:

- **soma_estoque**:
  - **Descrição**: responsável por iterar sobre os itens do vetor, chamar o procedimento `ajustar_preco` para cada item, somar o valor ajustado ao acumulador, e ao final retornar o valor total do estoque atualizado.
  - **Requisitos**:
    - Deve chamar internamente o procedimento `ajustar_preco`.
    - Recebe os seguintes argumentos:
      - `a0`: valor do item atual (vetor[i])
      - `a1`: endereço base do vetor
      - `a2`: índice do vetor (i)
      - `a3`: tamanho total do vetor
    - O resultado (valor total do estoque atualizado) deve ser retornado em a0

- **ajustar_preco**:
  - **Descrição:** Aplica o fator de aumento ao valor do item e, em seguida, chama o procedimento `aplicar_taxa` para calcular o valor final com o imposto.
  - **Requisitos:**
    - Deve chamar internamente o procedimento `aplicar_taxa`
    - Recebe os seguintes argumentos:
      - `a0`: valor original do item (vetor[i])
      - `a1`: fator de aumento
    - O resultado ajustado com a taxa deve ser retornado em `a0`.

- **aplicar_taxa**:
  - **Descrição**: aplica uma taxa de 50% sobre o valor recebido de `ajustar_preco` e retorna o resultado.
  - **Requisitos:**
    - Recebe os seguintes argumentos:
      - `a0`: valor original do item (vetor[i])
      - `a1`: valor da taxa de 50%
    - O valor final com a taxa aplicada deve ser retornado em `a0`

É **obrigatório** que o procedimento **`soma_estoque`** chame internamente o procedimento **`ajustar_preco`**, e que **`ajustar_preco`** chame **`aplicar_taxa`**. Trabalhos que não seguirem essa lógica terão nota zero, sendo feita uma conferência visual do código.

Este programa irá exercitar um conceito muito importante no assembly: a gestão da pilha de chamadas de subrotinas de forma correta, que é necessária para desenvolvermos programas em que um procedimento chama outro procedimento ou para que seja possível usar bibliotecas de terceiros. Em outras palavras, você terá que manipular o espaço da memória denominado *stack* (pilha de funções) para armazenar corretamente o endereço de retorno para as próximas instruções.

Utilize o esqueleto a seguir para o seu arquivo prob2.s (repare que a parte acima e abaixo do MODIFIQUE AQUI poderá ser alterada pelo professor/monitor no momento da correção).

```
.data

##### R1 START MODIFIQUE AQUI START #####
#
# Este espaço é para você definir as suas constantes e vetores auxiliares.
#

lista_precos: .word 7, 5, 1  # Vetor com os preços iniciais dos produtos
fator_aumento: .word 3       # Fator de aumentos do preço
taxa_50: .word 2             # Taxa adicional para ajustar o preço



##### R1 END MODIFIQUE AQUI END #####

.text

add s0, zero, zero     # Inicializa s0 com 0; s0 contará o número de testes que
passaram

li a0, 0               # Inicializa a0
la a1, lista_precos    # a1 aponta para o início da lista de preços
li a2, 0               # Índice inicial do vetor
li a3, 3               # Tamanho do vetor

jal ra, soma_estoque   # Chama função para somar todos os preços ajustados
addi t5, zero, 78      # Esperado: (7*3*2) + (5*3*2) + (1*3*2) = 42 + 30 + 6 = 78
bne a0, t5, teste_2    # Se o resultado for diferente, pula para teste_2
addi s0,s0,1           # Incrementa s0 (teste passou)

teste_2:

li a0, 3               # Inicializa a0
lw a1, fator_aumento   # a1 recebe o fator de aumento

jal ra, ajustar_preco  # Chama a função ajustar_preco(a0 = 3)
addi t5, zero, 18      # Valor esperado do resultado = 3 * 3 * 2 = 18
bne a0, t5, teste_3    # Se o resultado for diferente, pula para teste_3
addi s0,s0,1           # Incrementa s0 (teste passou)

teste_3:
li a0, 2               # a0 recebe 2, elemento i do vetor vetor[i]
lw a1, taxa_50       # a1 recebe a taxa de aumento

jal ra, aplicar_taxa   # Chama a função aplicar_taxa (a0 = 2, a3 = 2)
addi t5, zero, 4       # Esperado: 2 * 2 = 4
bne a0, t5, FIM        # Se diferente de 4, pula para FIM
addi s0,s0,1           # Incrementa s0 (teste passou)
j FIM

##### R2 START MODIFIQUE AQUI START #####

soma_estoque:

        # a0 elemento i do vetor, ou seja vetor[i]
        # a1 endereço do vetor
```

```
        # a2 elemento de incremento [i]
        # a3 tamanho do vetor

        jalr zero, 0(ra)

ajustar_preco:
        # a0 elemento i do vetor, ou seja vetor[i]
        # a1 fator de aumento

        jalr zero, 0(ra)

aplicar_taxa:
        # a0 elemento i do vetor, ou seja vetor[i]
        # a1 taxa de 50%

        jalr zero, 0(ra)

##### R2 END MODIFIQUE AQUI END #####

FIM:
```

Dicas e sugestões

- Não deixe o trabalho para o último dia. Não viva perigosamente!
- Comente seu código sempre que possível. Isso será visto com bons olhos.